COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	8/4/2026	**DATE SUBMITTED:**	8/20/2026
NAME:	Cedric S. Fernandez	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 4.1 **Stacks**

ILOB.2

CODE

MAIN.CC

```cpp
#include <iostream>
#include "stackList.h"

int main() {

    std::cout << "--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---\n";
    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>(); // Top should be 'e'
    displayALL<char>();
    std::cout << "\n ----- Popping all the elements -----\n";

    while(!isEmpty<char>()){
        std::cout << " Popped: " << pop<char>() << std::endl;
    }

    std::cout << "\n ------ Testing the Underflow ----- \n";
    pop<char>();

    return 0;
}
```

OUTPUT

```
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
Top of Stack: e
Displaying the stack:
e
p
c


 ----- Popping all the elements -----
 Popped: e
 Popped: p
 Popped: c

 ------ Testing the Underflow -----
stack overflow.
PS C:\Users\csf18\Desktop\DSA\HOA.4>
```

STACKLIST.H

stacklist.h

```
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
    public:
        T data;     // VALUE THAT WE NEED STORE
        Node* next; // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T>* head = nullptr;

//operations for the stack

// push
template <typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    //new node points to current top, then becomes the new head
    newNode->next = head<T>;
    head<T> = newNode;
}

// pop
template<typename T>
T pop(){
    //check if the stack is empty:
    if(head<T> == nullptr){
        std::cout<< "stack overflow. "<<std::endl;
    }

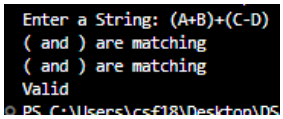
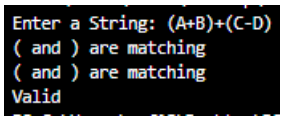
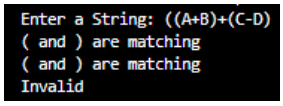
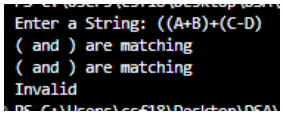
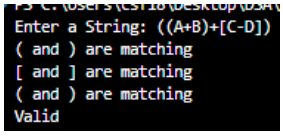
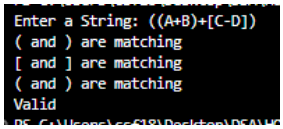
    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
    T tempVal = temp->data;

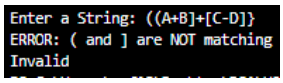
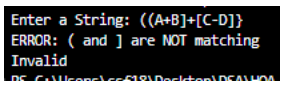
    // move the head to the next node
    head<T> = head<T>->next;
    //free the removed node
    delete temp;
    //return the removed value
    return tempVal;
}

// isEmpty
template<typename T>
bool isEmpty(){
    return head <T> == nullptr;
}

// top
template<typename T>
void Top(){
    // error catching:
    if(isEmpty<T>()){
        std::cout<<"Stack underflow "<<std::endl;
        return;
    }
    // display the Top node
    std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
}

//display All
template <typename T>
void displayALL(){
    if (head<T> == nullptr){
        std::cout << "stack undeflow\n";
        return;
        }
    std::cout<< "Displaying the stack: \n";
    Node<T>* temp = head<T>;
    while(temp != nullptr){
        std::cout << temp->data << "\n";
        temp = temp->next;
    }
    std::cout << "\n";
}
#endif
```

Supplementary			
Expression Valid	**(Y/N)**	**Output (Console Screenshot)**	**Analysis**
(A + B) + (C - D)	**Y**	**ARRAY STACK**  **LINK LIST STACK** 	For this run, the expression is balanced because every opening parenthesis has a matching closing symbol. As each opening bracket is pushed and then popped to verify a match with its corresponding closing pair, a complete match across all pairs confirms balance.
((A+B)+(C-D)	**N**	**ARRAY STACK**  **LINK LIST STACK** 	The expression is unbalanced because the stack remains non-empty at the end of execution, indicating that not all opening brackets were popped. Specifically, the initial opening parenthesis lacks a matching closing parenthesis.
((A+B)+[C-D])	**Y**	**ARRAY STACK**  **LINK LIST STACK** 	The expression is balanced because every opening symbol has a corresponding closing match. The innermost parentheses group A+B, the surrounding square brackets enclose C-D, and the outermost parentheses correctly encloses it entirely

((A+B]+[C-D]}	**N**	**ARRAY STACK**  **LINK LIST STACK** 	The expression is unbalanced because multiple symbols lack matching pairs. The closing square bracket next to B and the final closing brace } have no corresponding opening symbols, while the two opening parentheses at the beginning are never closed.

Tools Analysis:

How do the different internal representations affect the implementation and usage of the stack?

- An array based stack has a fixed capacity, limiting the total number of elements it can hold. In contrast, a linked list stack grows and shrinks dynamically without a rigid limit, treating the head node as the top of the stack. While templated linked-list functions are often defined in header files making the cpp file shorter, the primary difference between the two approaches is flexible memory allocation versus fixed size.

CONCLUSION

In this activity, I explored stack fundamentals by implementing both array based and linked list approaches. While choosing between them depends on whether the total number of elements is known beforehand, I found array stacks much easier to work with because they avoid the complex pointer management required by linked lists.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, code assistance and explanation of concept**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used to explain function concepts and how to properly use them, along with correcting my grammar for proper explanation and how to do circular Linked list**

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.